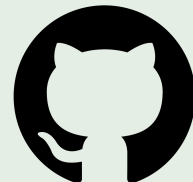
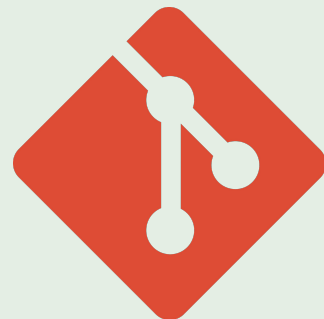


Corso Git con GitHub Desktop

*PER NON
PROGRAMMATORI*



GitHub

Firenze
2026



s.peruzzi@gmail.com



[@gundam79.bsky.social](https://bsky.social/@gundam79)



linkedin.com/in/stefanoperuzzi

Che cos'è Git?

- Git è un sistema che tiene traccia di tutte le modifiche che vengono effettuate ad una cartella, e a tutti i files in essa contenuti. La cartella in questione è detta **Repository**.
- Un tale sistema è chiamato VCS ossia Version Control System, sistema di controllo delle versioni.
- L'analogia è quella di una “macchina del tempo”: si può ritornare facilmente a qualsiasi versione precedente del progetto, senza perdere dati.
- Git è lo standard *de facto* per lo sviluppo di progetti software. Ma è utile in molti altri ambiti.



GitHub Desktop

GitHub è una piattaforma software che offre repository Git.

GitHub Desktop è lo strumento consigliato per usare Repository Git su GitHub.

Si scarica da <https://desktop.github.com/download/>

Si installa (già fatto?)

All'avvio ci si deve autenticare con un account GitHub.

Se non ce l'avete, createne uno (è gratis).

Configurate il vostro nome e la vostra email. 

GitHub permette di lavorare in più persone, **in modo collaborativo**, sullo stesso Repository.

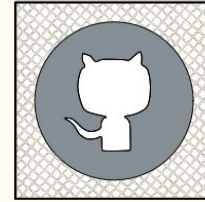
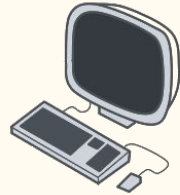
Come si usa Git

- Creazione di un repository
- Aggiunta di collaboratori
- **Clone** di un repository
- Modifica cartella: nuovi file, modifiche, cancellazioni
- **Commit**: registra nel Repository una “fotografia” di tutte le modifiche apportate.
- **push/pull**: sincronizza i commit del Repository locale con il Repository remoto

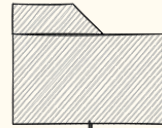
Locale

vs

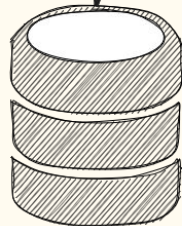
Remoto



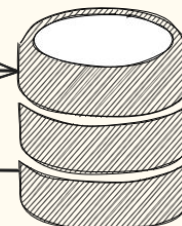
GitHub



commit



push



pull
(fetch)

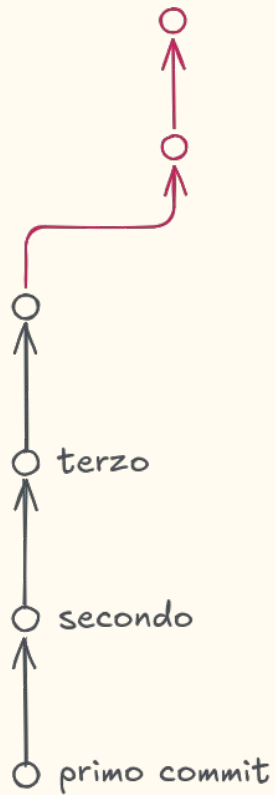
origin

Branches

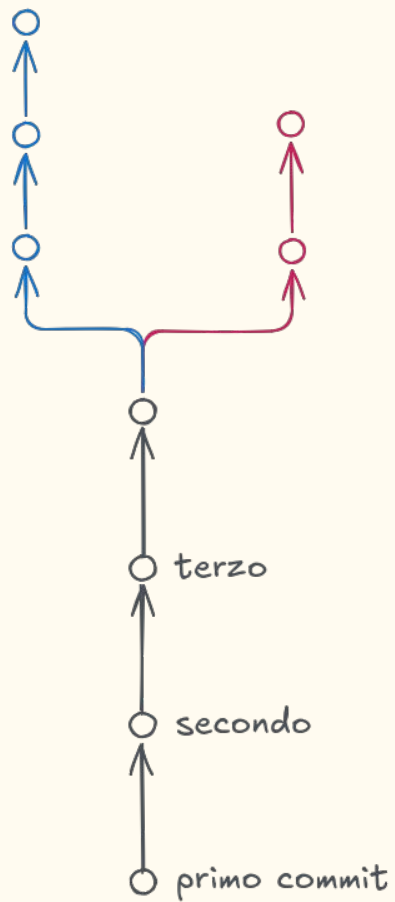
Il branch principale è
chiamato **main** per
convenzione



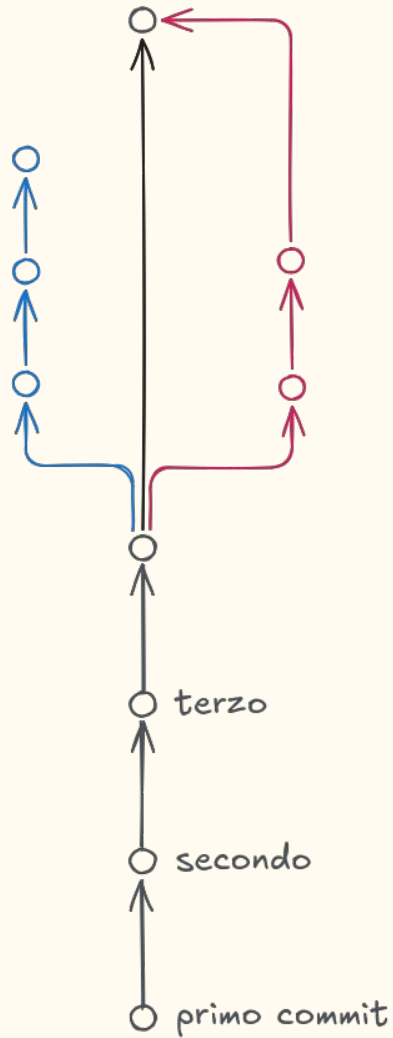
Branches



Branches



Branches



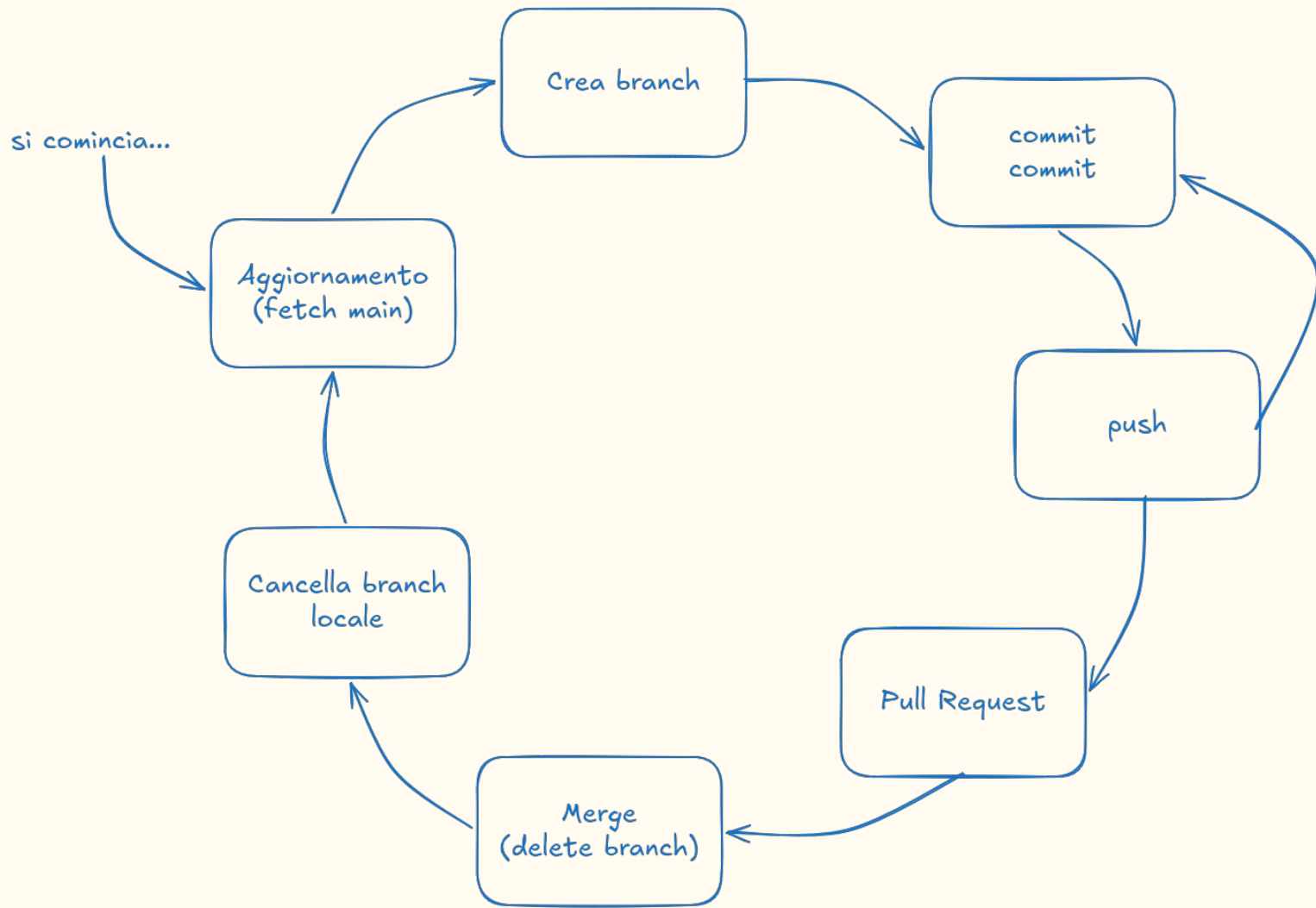
Pull Request

- Invece di eseguire un *merge*, chiediamo ad un nostro collaboratore di verificare il nostro lavoro, e di eseguire il merge del nostro branch sul branch *main*.
- La richiesta si chiama **Pull Request**.
- Il collaboratore può rifiutare la PR, o chiedere modifiche.
- Il processo garantisce che il branch *main* contenga lavoro approvato e completo.

Proviamo



Workflow



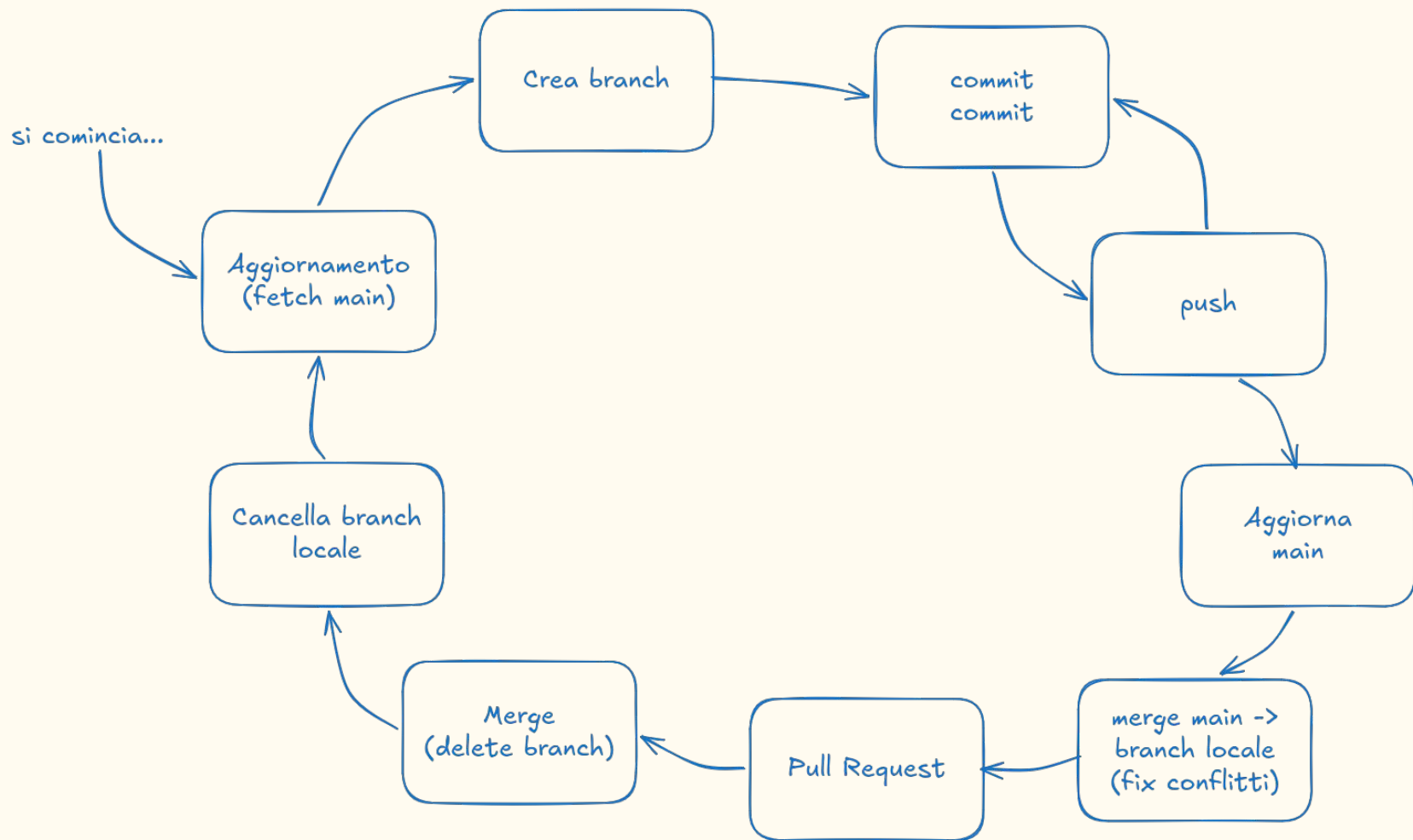
Conflitti

Legge di Murphy

«Se qualcosa può andare storto, lo farà»



Workflow esteso



E se ho sbagliato?

- Se ho eseguito un commit sbagliato e non l'ho ancora inviato (push):
 - History → Undo commit
 - Changes → Discard changes

E se ho sbagliato?

- Se ho eseguito un commit sbagliato e non l'ho ancora inviato (push):
 - History → Undo commit
 - Changes → Discard changes
- Per chi conosce Git CLI, sono le stesse operazioni:
 - `git reset HEAD~1`
 - `git reset --hard`

E se ho sbagliato?

- Se ho eseguito un commit sbagliato e l'ho già inviato su GitHub (push):
 - History → Revert changes in commit
- Se i commit “sbagliati” sono tanti?
 - History → checkout commit
 - Copia cartella git
 - Ritorno sul branch (HEAD)
 - Cancella cartella e ripristina contenuto
 - Commit “Ripristino versione del ... alle ...”

File da ignorare

Ci sono file che NON devono essere tracciati e inviati su GitHub:

- File temporanei (tmp, bak)
- File con configurazioni locali (.vscode, .DS_Store)

Per questi: **Changes** → **Ignore** / **Ignore all**

Viene aggiunta una voce al file **.gitignore** che fa parte del repo (commit+push)

Happy ~~coding~~ working with Git

